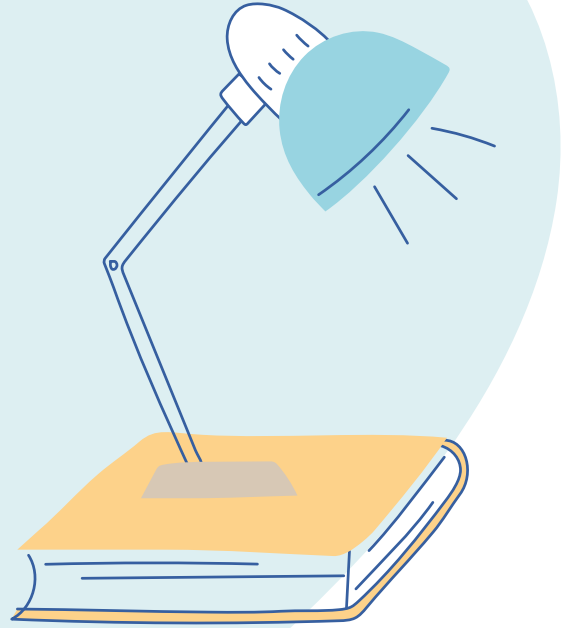


LU TALK

Myanmar Spelling Correction and Beyond

Presented by- Ei Thandar Phyu

23/08/2026



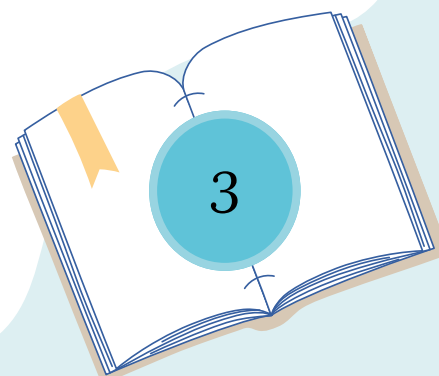
Context-Aware Myanmar Spelling Correction for Social Media Texts

Ei Thandar Phyu, Dr. Ye Kyaw Thu, Dr. Hutchatai Chanlekha, Dr. Thepchai Supnithi, Dr. Kotaro Funakoshi



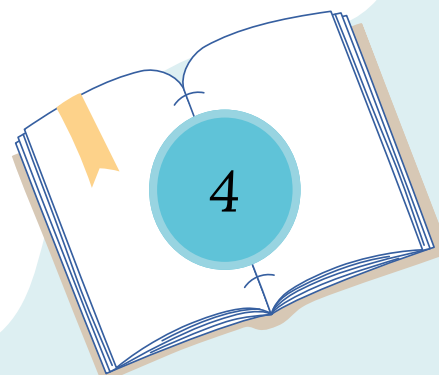
Introduction

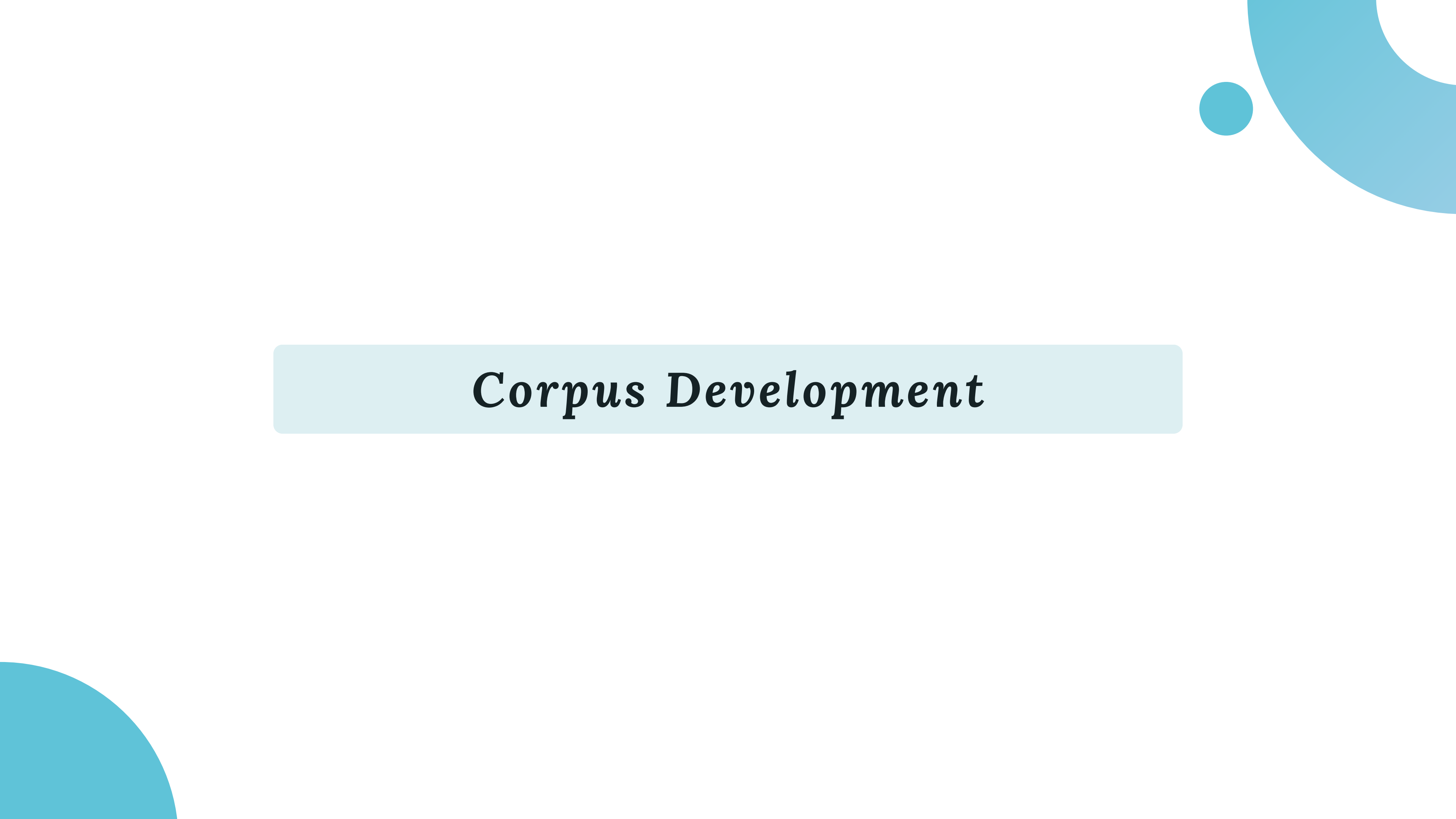
- Social media text often contains spelling errors and informal language, which can affect the clarity and interpretation of written communication.
- Spelling errors can also affect assistive technologies, such as Text-to-Speech (TTS) systems used by visually impaired users.
- The impact goes beyond human communication: spelling errors in input text can also reduce the performance of downstream NLP systems.
- For Myanmar, a low-resource language, developing such systems is particularly challenging because of limited language resources and spelling-error datasets.



Research Objectives

- To develop a context level Myanmar spelling correction system using neural machine translation techniques.
- To study the performance of spelling error correction through the integration of information about different types of errors into neural machine translation models.
- To study and evaluate the impact of the training set augmentation to the training process of neural machine translation models.
- To develop a spelling error-correction corpus specifically designed for the low-resource Myanmar language.



The slide features a light blue background with decorative teal elements. In the top right corner, there is a small teal circle and a larger teal arc. In the bottom left corner, there is a large teal arc. A horizontal teal bar with rounded ends is positioned in the center of the slide.

Corpus Development

Myanmar Spelling Error Types

- Consonant Error (con)

- ရောက်ရှိ → ကောက်ဝံ (ရ → ဝ)

- Dialect Error (dialect)

- ပိုက်ဆံအိတ် → ပတ်တာအိတ်, ပတ္တာအိတ်

- Phonetic Error (pho)

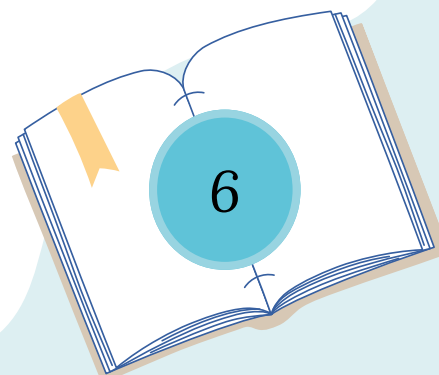
- ပတ်သက် → ပက်သက်, ပက်သတ်, ပတ်သတ်

- Sensitive Error (sensitive)

- ဖင် → ဖျင်, ဘင်, *င်, ဖင်, -င်, phင်, phin

- Sequence Error (seq)

- အံ့သြ <အ + ဝံ့ + ဝံ့ + သြ> → အံ့သြ <အ + ဝံ့ + ဝံ့ + သြ>



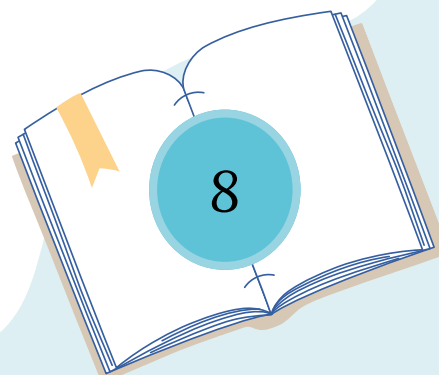
Myanmar Spelling Error Types

- **Short-form Error (short)**
 - အတူတူ → အတူ၂, အတူ၃, အတူ”
- **Slang Word Error (slang)**
 - ပိုက်ဆံ → ပိုက်ပိုက်, ပိုက်ချုံ, ပတ်ချုံ
- **Stack Word Error (stack)**
 - မဏ္ဍပ် → မဏ္ဍပ်, မဏ္ဍပ်, မဏချပ်, မဏပ်
- **Encoding Error (encode)**
 - သင်္ချာ → သချာ
- **Typographic Error (typo)**
 - ကျေးဇူးတင်ပါတယ် → ကျွေးဇူးတင်ပါတယ်



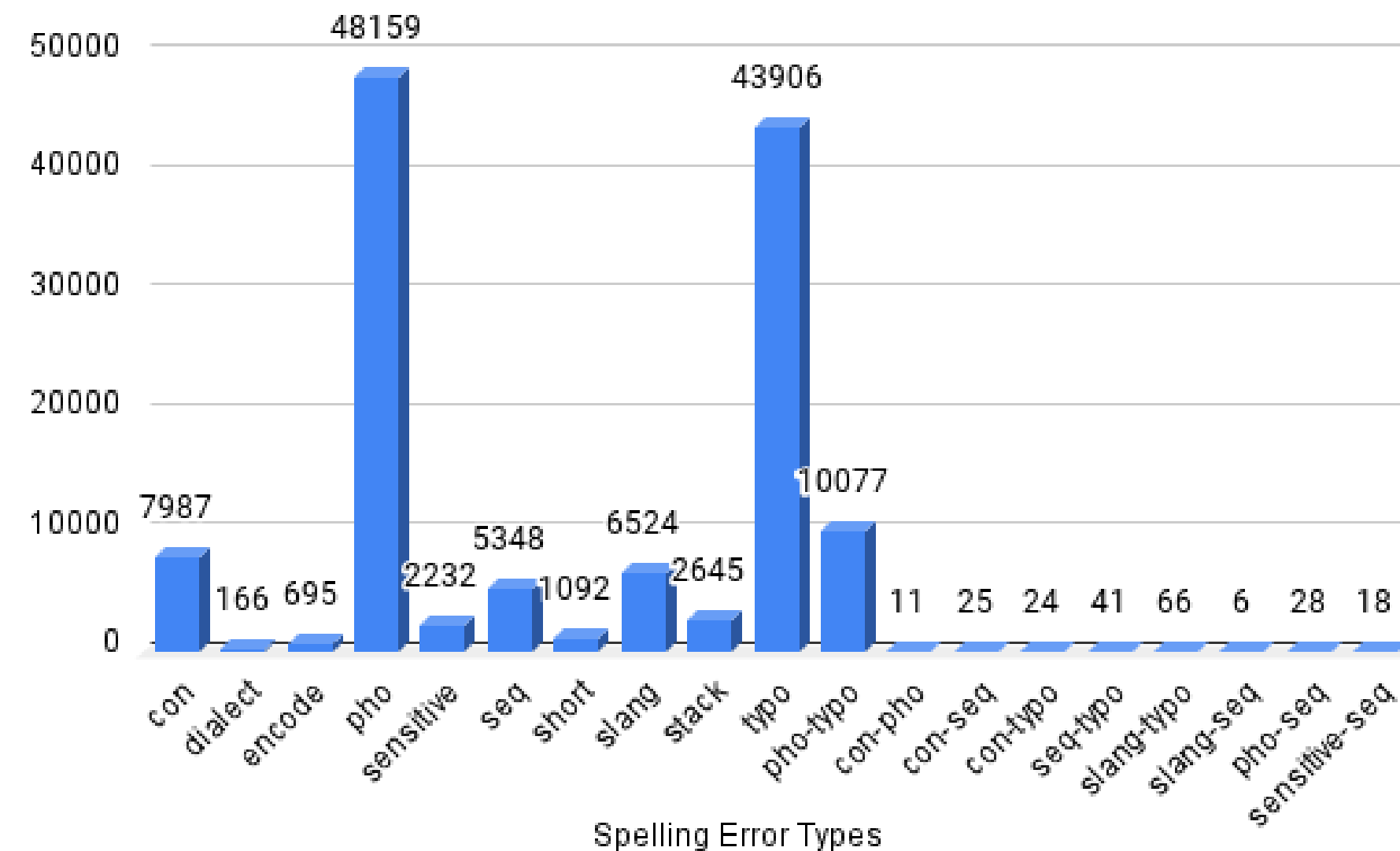
Corpus Development

- Burmese spelling training corpus for the “**SymSpell4Burmese**”, is developed by Ei Phyu Phyu Mon et. al. [1], with 22,850 sentences.
- We extended that corpus and developed the corpus which comprises 63,036 sentence/phrase pairs in total.
- The spelling error sentences are collected by manually collecting texts from social media platforms, news outlets, and blogs.
- The misspelled syllables are manually annotated according to its error types.
 - <◌[◌]◌|sensitive>
- When the syllable falls into two error categories, it is tagged as belonging to both.
 - <◌[◌]◌|pho-typo>



Corpus Development

- Error sentence/phrase was distinguished from the corrected sentence/phrase by three pipes “|||”.
 - <မင်|stack> လာပါ<ရှင်|con>||| <မင်> လာပါ<ရှင်>||
 (“Hello.” in English)

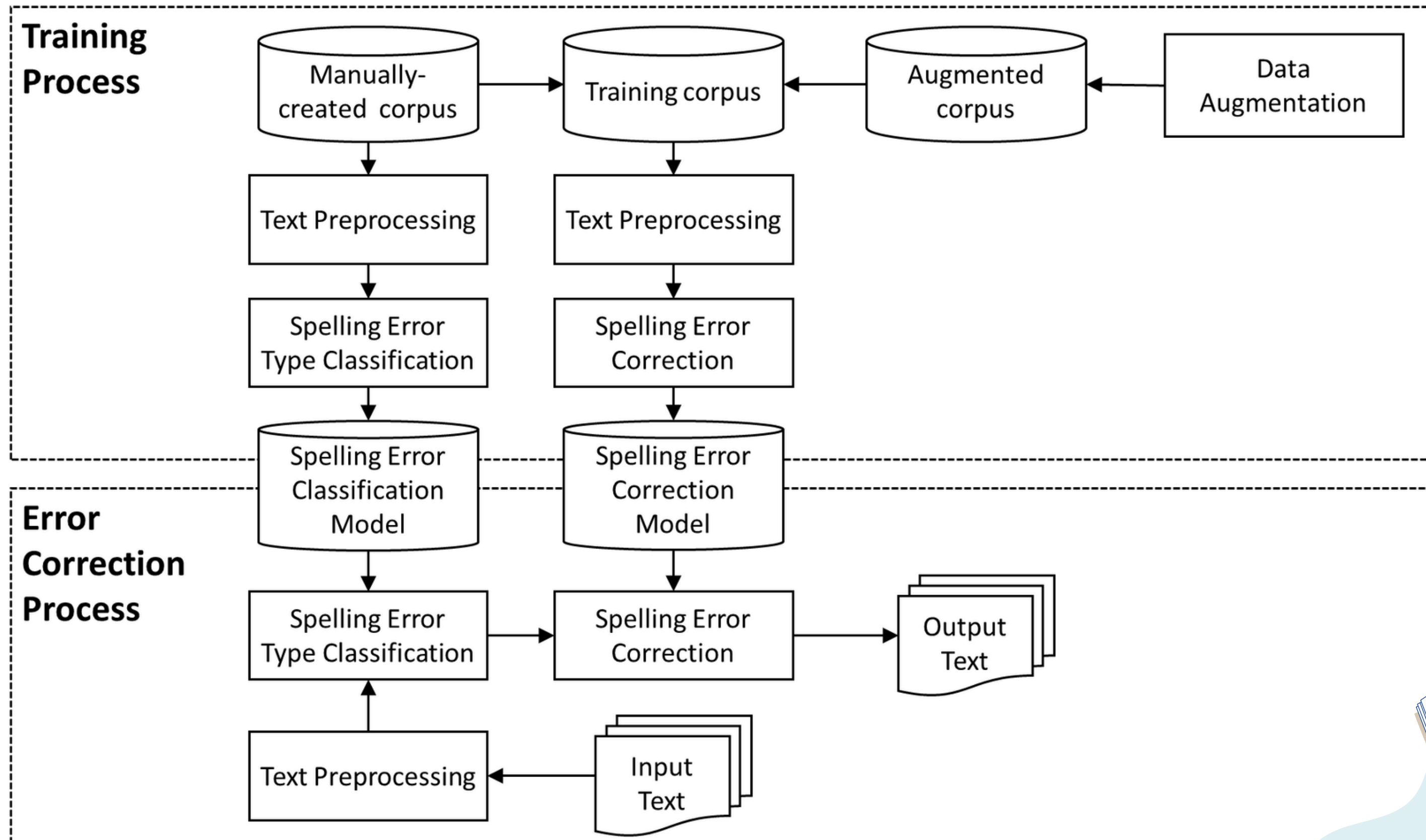


Occurrence of Spelling Errors by Error Types



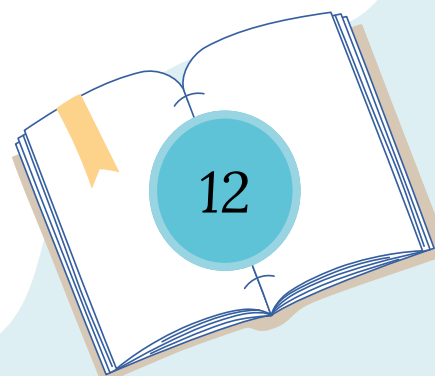
Methodology

Methodology

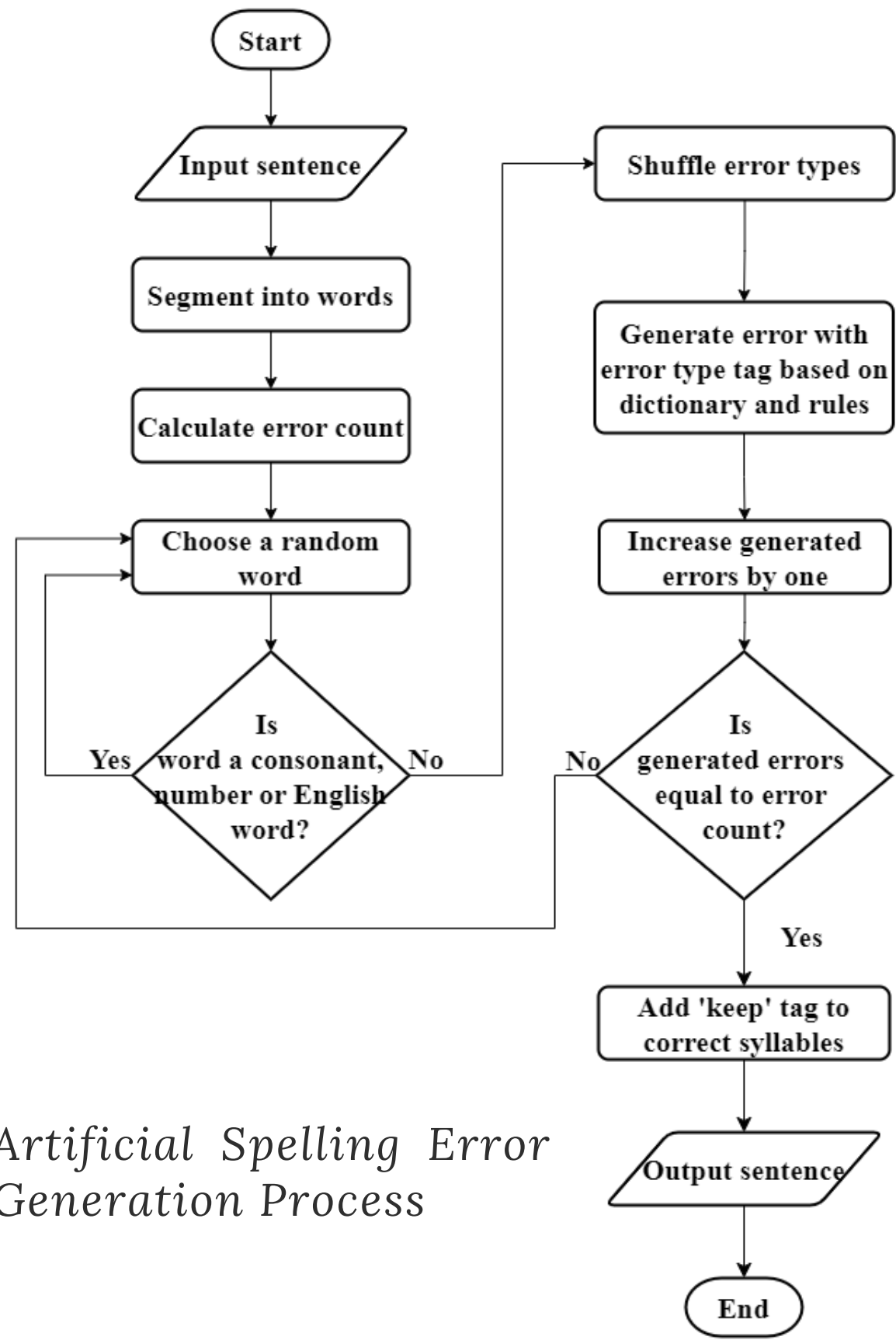


Data Augmentation

- **Dictionaries and rules** are created for each error type based on the most common error patterns in the real world data.
- **Typo errors** are simulated by replacing character with nearby ones on the keyboard, inserting, deleting or duplicating characters.
- **Phonetic errors** are generated by replacing syllables with randomly chosen ones from a dictionary containing syllables sharing the same phoneme.
 - e.g., chin ['ချင်', 'ချဉ်', 'ခြင်']; za- ['စ', 'ဇ']; nju' ['ညွတ်', 'ညွတ်']
- **Other error types** have their dictionaries containing common spelling mistakes and corresponding rules for replacement.
 - e.g., **Rule for short-form error**, (if two consecutive syllables in a word are same, replace the 2nd syllable with 2, ၂ or ”)
 - e.g., **Dictionary for consonant error**, 'ရ': ['၇']; 'ဂ': ['စ']; 'သ': ['သ', 'သ']



Data Augmentation



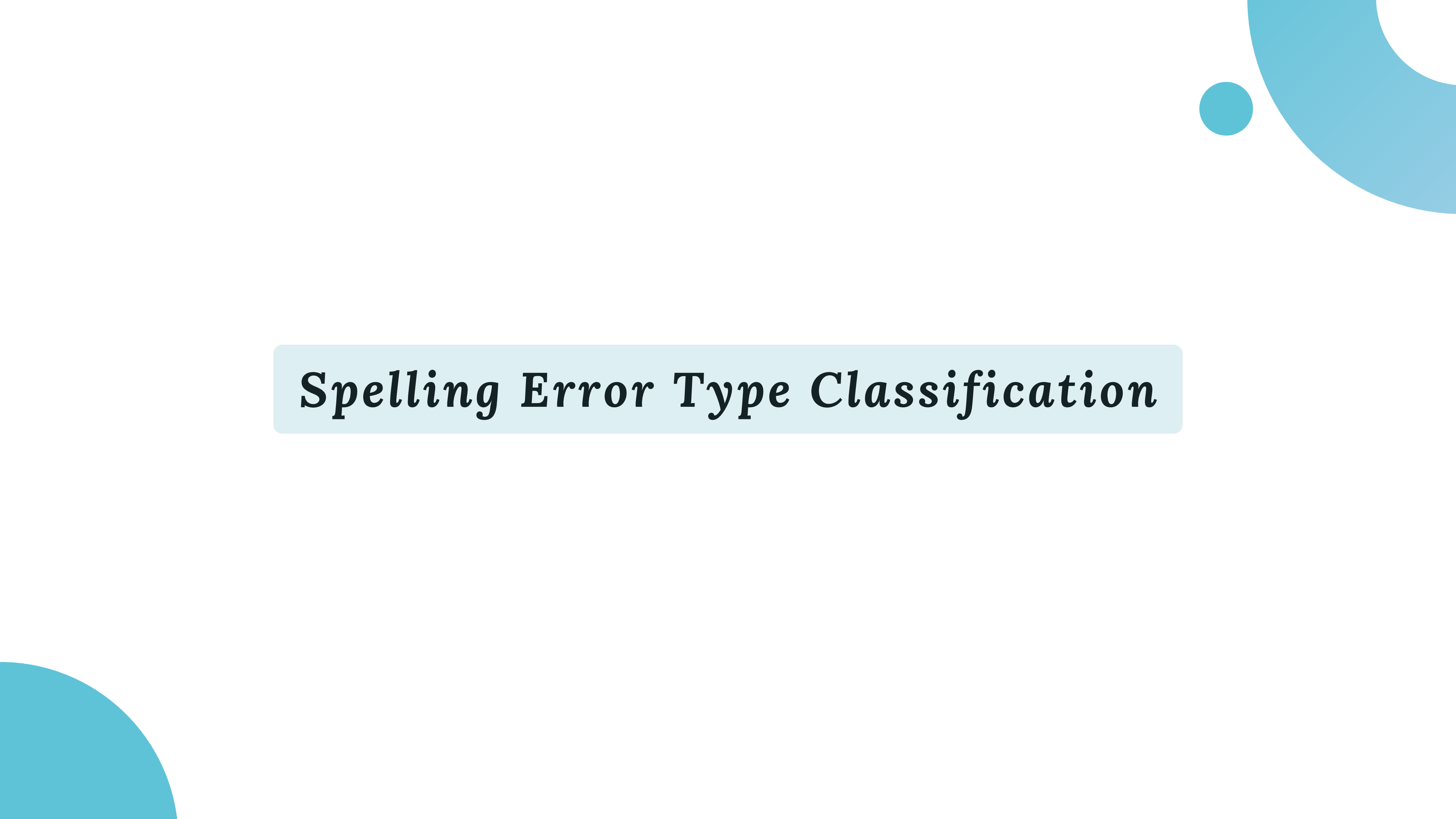
Artificial Spelling Error Generation Process

- Example 1:

- Input: ကမ္ဘာ ကြီး က လုံး တယ်
- error count: 1 (Simulate errors in 20% of the sentence.)
- Output: ကမ္ဘာ|keep ကြီး|keep က|keep လုံး|keep တရ်|slang

- Example 2:

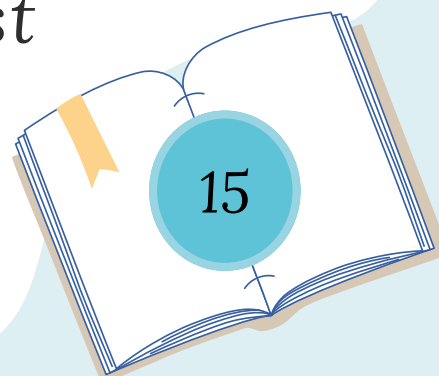
- Input: လမ်းပိတ် နေ လို့ မြေအောက် ရထား စီး တာ ပို ကောင်း လိမ့် မယ်
- error count: 3 (Simulate errors in 20% of the sentence.)
- Output: လမ်း|keep ပိတ်|keep နေ|keep ရုံ|slang မြေ|keep အောက်|keep ဂ|con ထား|keep စီး|keep တာ|keep ပို|keep ကော်|typo လိမ့်|keep မယ်|keep



Spelling Error Type Classification

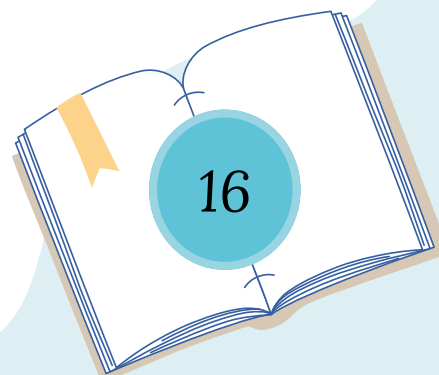
Spelling Error Type Classification

- Spelling error type classification is the process of categorizing the misspelled words into specific types.
- We explored three approaches for Myanmar spelling error type classification:
 - **Tsetlin Machine**
 - a general-purpose, interpretable, and low-energy ML approach [3] .
 - **fastText**
 - a library for learning of word embeddings and text classification created by Facebook's AI Research (FAIR) lab [4],
 - **BiLSTM**
 - an extension of LSTM that processes sequences in both forward and backward directions, enabling the model to capture context from both past and future tokens.

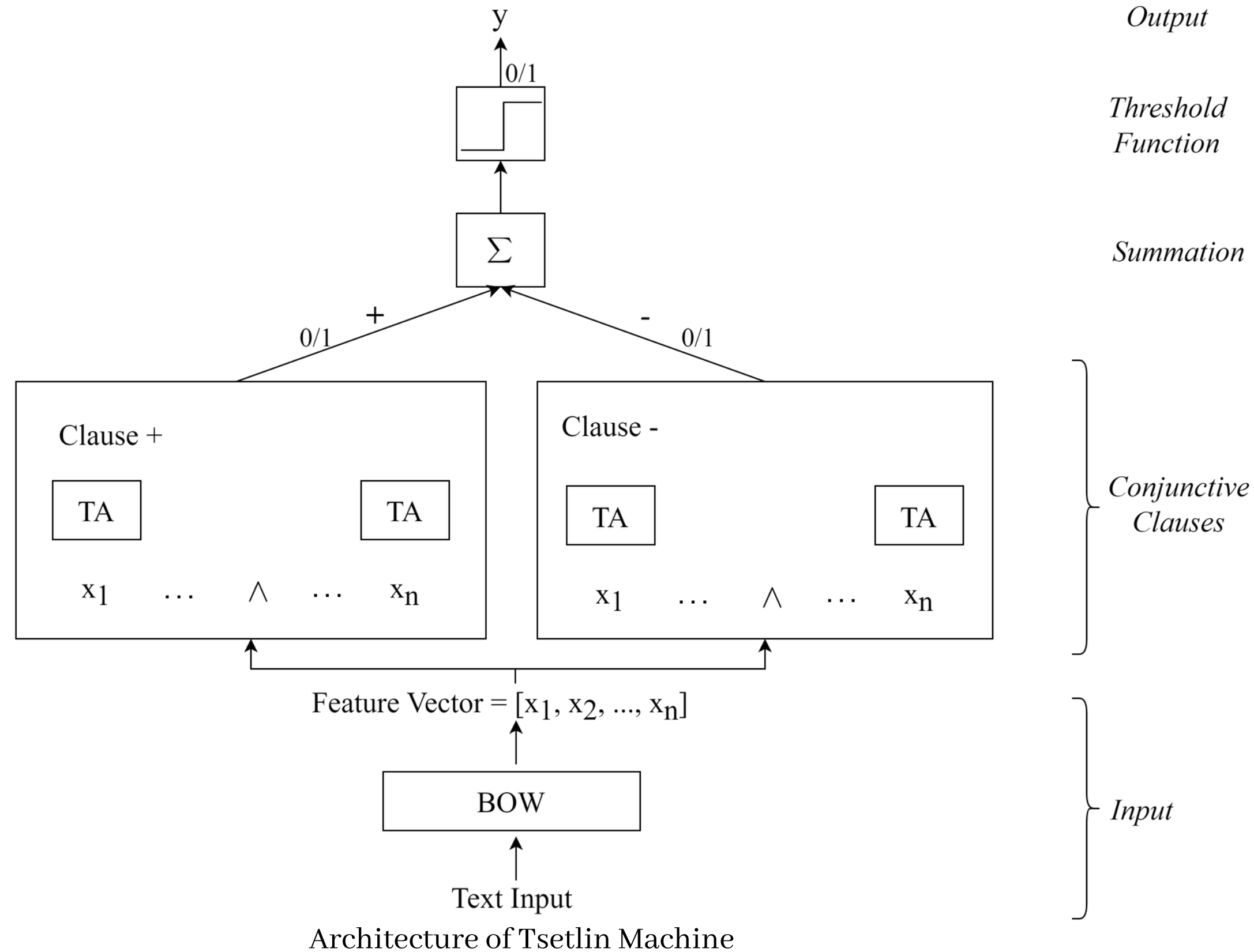


Related Theory: Tsetlin Machine

- The Tsetlin machine, introduced by Ole-Christofer Granmo in 2018, is a general-purpose, interpretable, and low-energy machine learning approach.
- It breaks down problems into self-contained patterns, each of which is represented as conjunctive clauses in propositional logic.
- It incorporates bandit learning, a method for learning the optimal action in an environment from penalties and rewards.
- It has promising properties in terms of computational simplicity, transparency and interpretability.

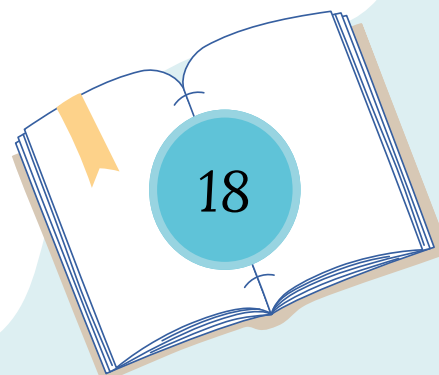


Related Theory: Tsetlin Machine



Related Theory: fastText

- *fastText is a library for learning of word embeddings and text classification created by Facebook's AI Research (FAIR) lab.*
- *It supports supervised (classifications) and unsupervised (embedding) representations of words and sentences.*
- *It incorporates a bag of n-grams representation along with word vectors to preserve some information about the surrounding words appearing near each word.*
- *It supports hierarchical softmax as an activation function which can highly reduced order of complexity and speed up the classification process.*



Related Theory: LSTM

Long Short-Term Memory (LSTM) and Bidirectional LSTM

- LSTM [3] is a type of Recurrent Neural Network designed to overcome the vanishing gradient problem, making it effective for learning long-term dependencies in sequential data.
- This is useful for language because whether something is an error can depend not only on the word itself but also on the surrounding sequence.
- BiLSTM extends the LSTM by processing sequences in both forward and backward directions, capturing context from both past and future information.
- LSTM models are widely used for text classification, machine translation, and sentiment analysis.



Experiments: Error Type Classification

Tsetlin Machine

- Corpus: 107,935 n-grams sequences
 - 97,141 sequences for training and 10,794 for testing
- Input format:
 - sensitive မှု ဝေး _ သဲ သေး
 - short ကော်^၂ များ

fastText

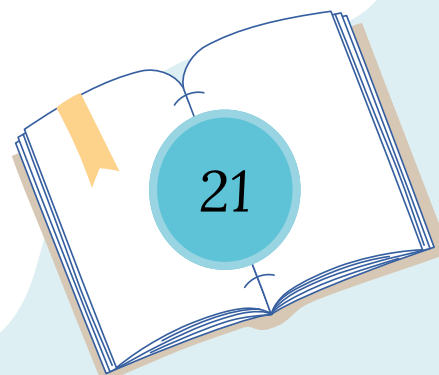
- Corpus: 107,935 n-grams sequences
 - 97,141 sequences for training and 10,794 for testing
- Input format:
 - __label__sensitive မှု ဝေး _ သဲ သေး
 - __label__short ကော်^၂ များ



Experiments: Error Type Classification

BiLSTM

- Corpus: 63,036 erroneous sentences/phrases
 - 44,126 sentences/phrases for training, 9,456 for validation, and 9,456 for testing.
- Input format:
 - မှန်/stack မှန်/keep မှန်/keep မှန်/con



Result: Error Type Classification

Testlin Machine

fit_epoch	Precision	Recall	F1 Score
5	0.63	0.57	0.59

Precision, Recall and F1 Score of Tsetlin Machine

- The models are configured with **20 clauses** and trained for **100 iterations** over the entire training dataset.
- We conducted the following experiments to identify optimal hyperparameters:
 - **n-gram range:** (1, 2), (1, 3), and (1, 4)
 - **T value:** from 10 to 50 in increments of 5
 - **s value:** 3.9, 5, and 8
 - **Inner training epochs:** 1, 3, and 5 epochs
- The highest scores were achieved with a T of 15, an s of 1.5, a CountVectorizer using an n-gram range of (1, 3), and 5 inner training epochs.



Result: Error Type Classification

fastText

Epochs	Precision	Recall	F1 Score
20	0.842	0.781	0.810

Precision, Recall and F1 Score of fastText

- We found that the precision, recall, and F1 score had a consistent pattern over a range of **training epochs from 10 to 100**.
- The similarities in scores between the epochs indicate how early in the training process the model was able to converge to a stable state.



Result: Error Type Classification

BiLSTM

Precision	Recall	F1 Score
0.8297	0.8580	0.8436

Precision, Recall and F1 Score of BiLSTM

- The BiLSTM model achieved a good balance between precision and recall.
- This results in a high F1-score, making BiLSTM the most effective approach for handling Myanmar error type classification.
- Since BiLSTM model achieved the best results, we applied BiLSTM based error type classifier to annotate spelling error type.

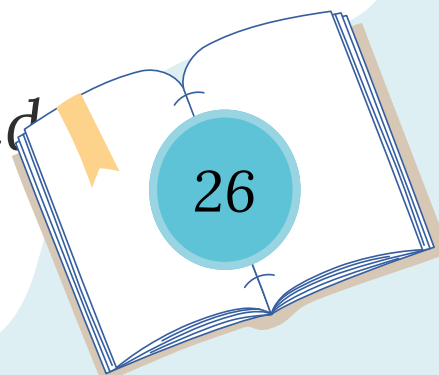




Spelling Error Correction

Spelling Error Correction

- We treated the Myanmar spelling correction system as machine translation process utilizing the sentence-wise NMT approach.
- In this study, we explored two NMT based models:
 - **Sequence-to-sequence with LSTM**
 - a model that uses an encoder-decoder framework: the encoder compresses the input into a context vector, and the decoder generates the output sequence.
 - **Transformer**
 - a model that uses self-attention to process sequences efficiently, enabling parallelization and strong NLP performance.
- We also investigated the performance of **integrating error type information and data augmentation** in our models.



Experiments: Spelling Error Correction

- For the implementation of our proposed spelling correction models, we used the openNMT framework [8], an open source toolkit to train NMT models.
- To train the model without incorporating error type features, all error type tags will be eliminated.
 - Source: မင်္ဂလါပိုင်
 - Target: မင်္ဂလါပိုင်

	No. of Sentence	No. of Spelling Mistakes
Training	44,126	90,423
Validation	9,455	19,387
Testing	9,455	19,240

Experiments: Spelling Error Correction

Incorporating Error Type Information: Seq2Seq with LSTM

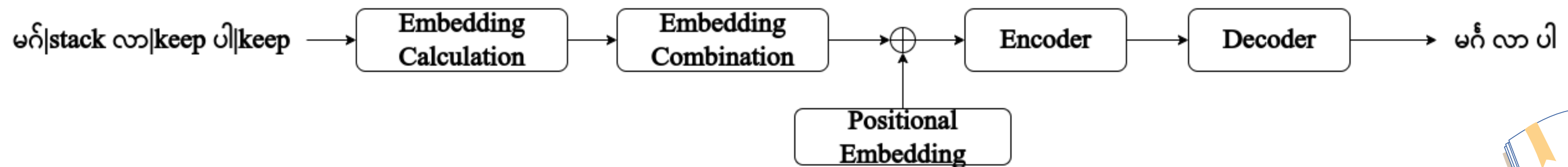
- Unlike the standard model, the encoder of the multi-input feature model [9] incorporates additional input features by using a combined embedding vector that integrates these additional features.
- Concatenation, summation, or a multi-layer perceptron (MLP) can be used to combine the source text and feature embeddings.
- The length of the combined embedding vector matches the total embedding size, and all other parts of the model remain unchanged.
- In this study, we refer to the sequence-to-sequence model with LSTM that incorporates additional input features as “**Seq2Seq-LSTM+**”.



Experiments: Spelling Error Correction

Incorporating Error Type Information: Transformer

- In a multiple-input feature Transformer model, additional input features are integrated through a combined embedding [10].
- The positional encoding is then summed with the combined vector and fed into the encoder.
- We implemented a Transformer-based spelling correction model by adding the error type feature and denoted it as “**Transformer+**”.

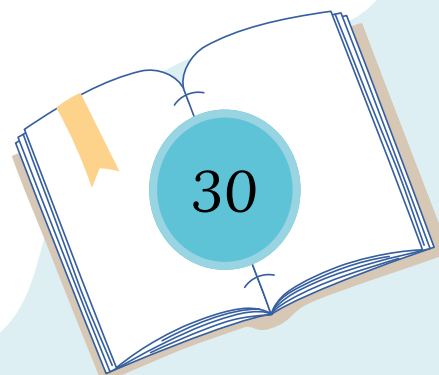


Architecture of the “Transformer+” Model

Experiments: Spelling Error Correction

Incorporating Error Type Information

- To train the ‘Seq2Seq-LSTM+’ and ‘Transformer+’ model, additional error type information are appended to the source sentence using a feature separator, ‘|’.
- A “keep” tag was utilized to indicate the correct spelling in the source (erroneous) sentence.
- Example:
 - Original: <မင်|stack>လာပါ<ဂွံ|con>။ ||| <မင်>လာပါ<ရှံ>။
 - Preprocessed_err: မင်|stack လာ ပါ ဂွံ|con
 - Preprocessed_crr: မင် လာ ပါ ရှံ
 - Source: မင်|stack လာ|keep ပါ|keep ဂွံ|con
 - Target: မင် လာ ပါ ရှံ



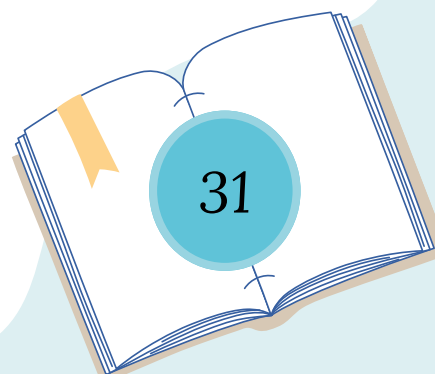
Result: Spelling Error Correction

Experiments to Identify Optimal feat_merge Options in Seq2Seq-LSTM

	Precision	Recall	F1 score	BLEU	GLEU	WER
Seq2Seq-LSTM	0.6973	0.7263	0.7115	0.9308	0.9623	3.9
Seq2Seq-LSTM+ (feat_merge: sum)	0.8178	0.8288	0.8233	0.9615	0.9758	2.3
Seq2Seq-LSTM+ (feat_merge: concat)	0.8713	0.8333	0.8519	0.9641	0.9765	2
Seq2Seq-LSTM+ (feat_merge: mlp)	0.6188	0.7798	0.6901	0.9401	0.969	4.6

Performance Metrics for Spelling Error Correction applying '**Seq2Seq LSTM**' and '**Seq2Seq-LSTM+**' with three 'feat_merge' options

*Manually annotated error types were used in the testing data to determine the upper bound of the model's performance.



Result: Spelling Error Correction

Experiments to Identify Optimal Augmentation Level in Seq2Seq-LSTM+ with feat_merge: concat

Augmentation Level	Precision	Recall	F1 score	BLEU	GLEU	WER
0%	0.8713	0.8333	0.8519	0.9641	0.9765	2
25%	0.8786	0.8384	0.8584	0.9651	0.9777	2
50%	0.9047	0.846	0.8744	0.9673	0.9786	1.8

Performance Metrics for Spelling Error Correction using ‘Seq2Seq-LSTM+’ with integration of error type features with ‘mlp’ feat_merge option and data augmentation

*Manually annotated error types were used in the testing data to determine the upper bound of the model's performance.



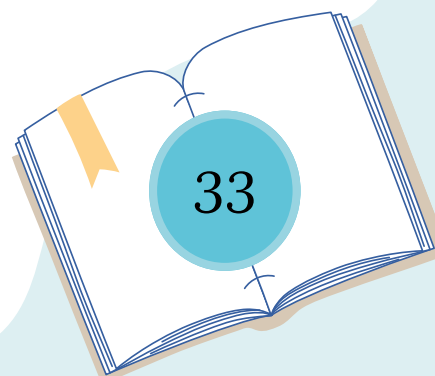
Result: Spelling Error Correction

Experiments to Study the Performance of Seq2Seq-LSTM+ with Automatic Classifier

	Precision	Recall	F1 score	BLEU	GLEU	WER
Seq2Seq-LSTM	0.6973	0.7263	0.7115	0.9308	0.9623	3.9
Seq2Seq-LSTM+	0.7838	0.7468	0.7648	0.9377	0.9644	3.3
Seq2Seq-LSTM+ (with augmentation)	0.8061	0.7542	0.7793	0.9403	0.9660	3.1

Performance Metrics for Spelling Error Correction utilizing '**Seq2Seq-LSTM**' and '**Seq2Seq-LSTM+**' with BiLSTM-based Error Type Classification Model

*Error types annotated by the BiLSTM-based automatic classifier were used in the testing data, considering the potential for error propagation within our correction model.



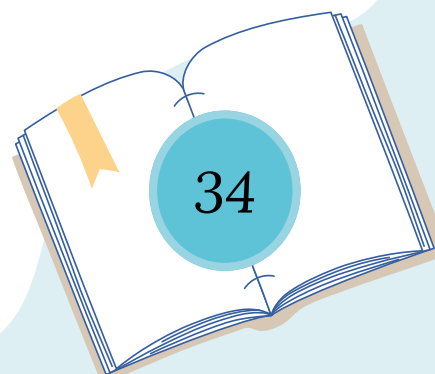
Result: Spelling Error Correction

Experiments to Identify Optimal feat_merge Options in Transformer

	Precision	Recall	F1 score	BLEU	GLEU	WER
Transformer	0.7926	0.7283	0.7591	0.9357	0.9648	3.3
Transformer+ (feat_merge: sum)	0.9048	0.8324	0.8671	0.9624	0.9766	2
Transformer+ (feat_merge: concat)	0.9181	0.834	0.874	0.963	0.9769	1.9
Transformer+ (feat_merge: mlp)	0.9159	0.8359	0.8741	0.9637	0.9769	1.8

Performance Metrics for Spelling Error Correction applying '**Transformer**' and '**Transformer+**' with three 'feat_merge' options

*Manually annotated error types were used in the testing data to determine the upper bound of the model's performance.



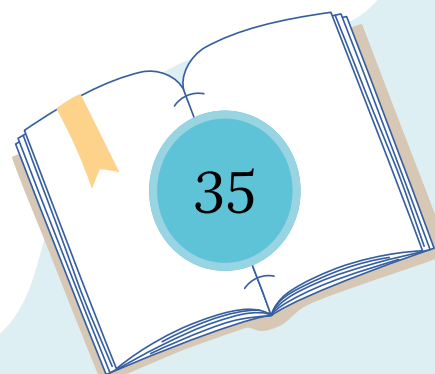
Result: Spelling Error Correction

Experiments to Identify Optimal Augmentation Level in Transformer+ with feat_merge: mlp

Augmentation Level	Precision	Recall	F1 score	BLEU	GLEU	WER
0%	0.9159	0.8359	0.8741	0.9637	0.9769	1.8
25%	0.9093	0.8419	0.8743	0.9647	0.9778	1.8
50%	0.9118	0.8399	0.8744	0.9648	0.9781	1.8

Performance Metrics for Spelling Error Correction using ‘**Transformer+**’ with integration of error type features with ‘mlp’ feat_merge option and data augmentation

*Manually annotated error types were used in the testing data to determine the upper bound of the model's performance.



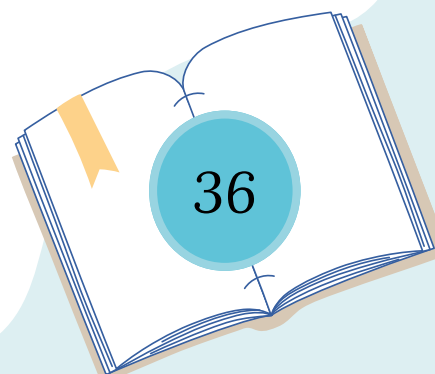
Result: Spelling Error Correction

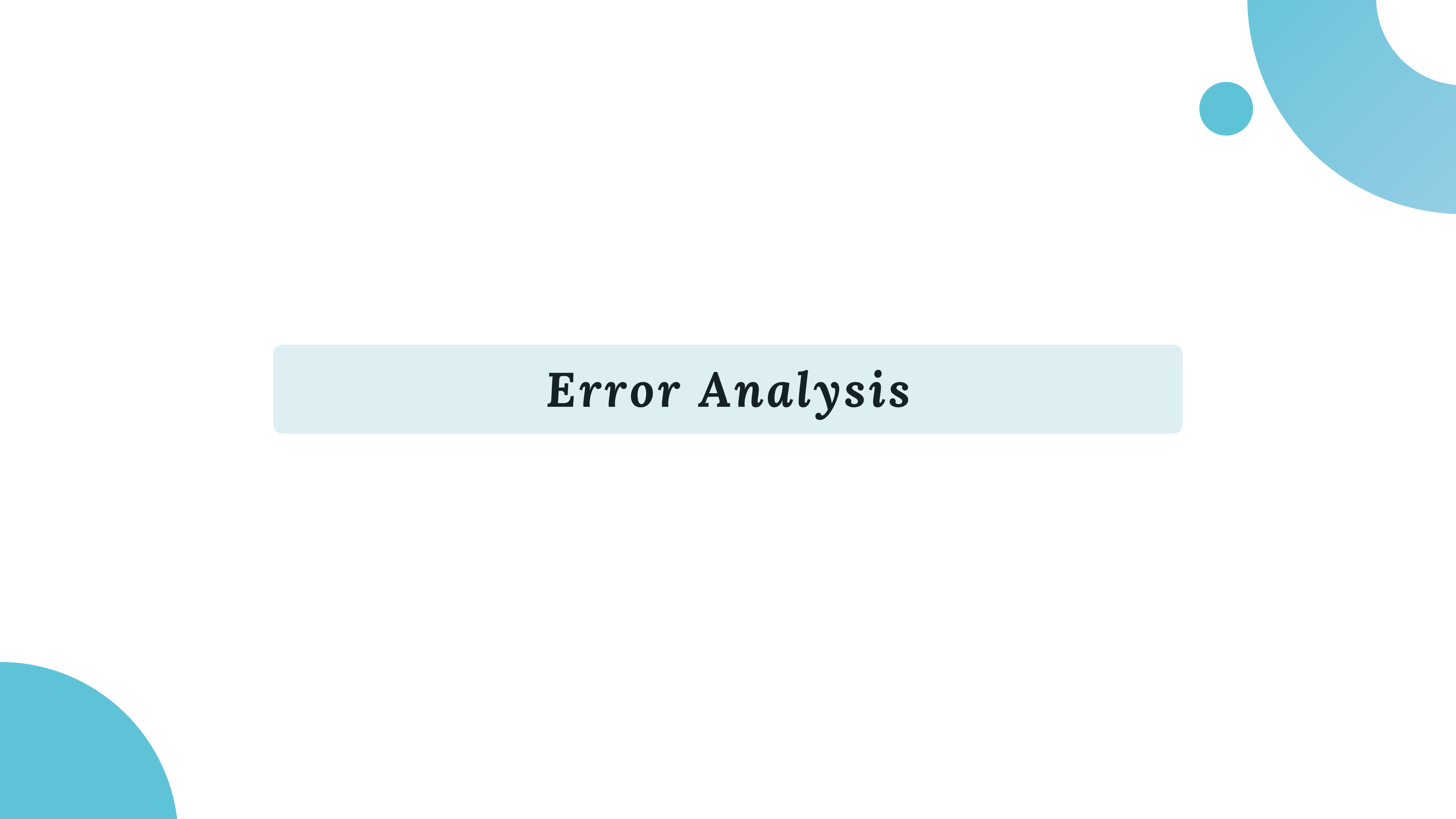
Experiments to Study the Performance of Transformer+ with Automatic Classifier

	Precision	Recall	F1 score	BLEU	GLEU	WER
Transformer	0.7926	0.7283	0.7591	0.9357	0.9648	3.3
Transformer+	0.7907	0.7443	0.7668	0.9367	0.9641	3.2
Transformer+ (with augmentation)	0.8264	0.7471	0.7847	0.9369	0.9650	3.1

Performance Metrics for Spelling Error Correction utilizing ‘**Transformer**’ and ‘**Transformer+**’ with BiLSTM-based Error Type Classification Model

*Error types annotated by the BiLSTM-based automatic classifier were used in the testing data, considering the potential for error propagation within our correction model.





Error Analysis

Error Analysis

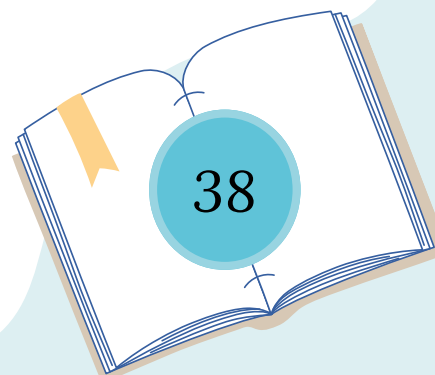
- We performed a detailed error analysis on the outputs of ‘Seq2Seq-LSTM’, ‘Transformer’, ‘Seq2Seq-LSTM+’ and ‘Transformer+’ testing with automatically annotated error labels.

- Ex.1

- Source: များ ရိုး စား နော်
- Reference: အ များ ကြီး စား နော်
- Seq2Seq-LSTM: များ ရည်း စား နော်
- Seq2Seq-LSTM+: အ များ ကြီး စား နော်
- Transformer: များ ရည်း စား နော်
- Transformer+: အ များ ကြီး စား နော်

- Ex.2

- Source: လူ တိုင် က အ မှာ နဲ့ မ ကင် ပါ ဘူး
- Reference: လူ တိုင်း က အ မှား နဲ့ မ ကင်း ပါ ဘူး
- Seq2Seq-LSTM: လူ တိုင်း က အ မှာ နဲ့ မ ကင် ပါ ဘူး
- Seq2Seq-LSTM+: လူ တိုင်း က အ မှား နဲ့ မ ကင်း ပါ ဘူး
- Transformer: လူ တိုင်း က အ မှာ နဲ့ မ ကင်း ပါ ဘူး
- Transformer+: လူ တိုင်း က အ မှား နဲ့ မ ကင်း ပါ ဘူး



Error Analysis

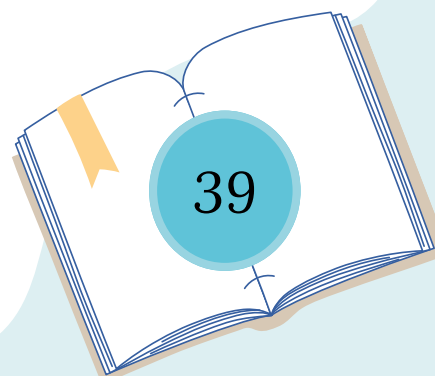
- We analyzed the corrected outputs of spelling correction models by extracting the confusion pairs using SCLITE toolkit.

Confusion Pairs of Seq2Seq-LSTM

Frequency	Reference >> Output
49	တ >> တစ်
43	ပြီ >> ပြီး
40	ကိုယ် >> ကို
39	တက် >> တတ်
33	ပြီး >> ပြီ

Confusion Pairs of Seq2Seq-LSTM+

Frequency	Reference >> Output
40	တစ် >> တ
37	တက် >> တတ်
34	ကိုယ် >> ကို
33	တ >> တစ်
32	လို >> လို့



Error Analysis

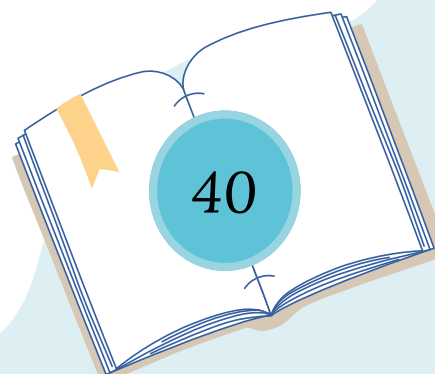
- We analyzed the corrected outputs of spelling correction models by extracting the confusion pairs using SCLITE toolkit.

Confusion Pairs of Transformer

Frequency	Reference >> Output
39	လို့ >> လို
37	တ >> တစ်
36	ကို >> ကိုယ်
36	ပြီး >> ပြီ
32	ဲ >> ဘဲ

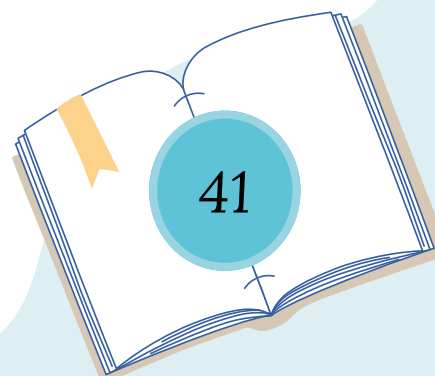
Confusion Pairs of Transformer+

Frequency	Reference >> Output
40	တစ် >> တ
35	တက် >> တတ်
34	ကိုယ် >> ကို
31	ကျ >> ကြ
31	တ >> တစ်



Error Analysis

- When analyzing the SCLITE outputs for the Seq2Seq-LSTM model, we observed that the model struggles with handling long sequences.
- When three long sentences are tested, the score outputs are as follows:
 - 1.Scores: (#C #S #D #I) 116 26 23 25
 - 2. Scores: (#C #S #D #I) 107 35 2 3
 - 3. Scores: (#C #S #D #I) 112 43 4 1
 - #C : The number of corrected outputs from the models.
 - #S: The number of incorrect substitution.
 - #D: The number of incorrect deletion.
 - #I: The number of incorrect insertion.

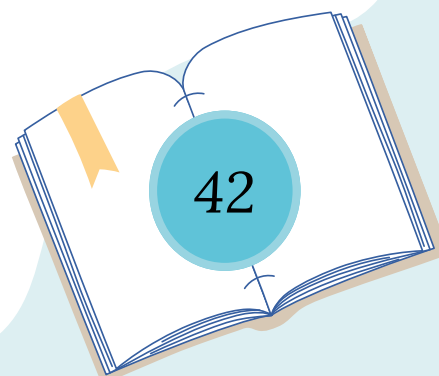


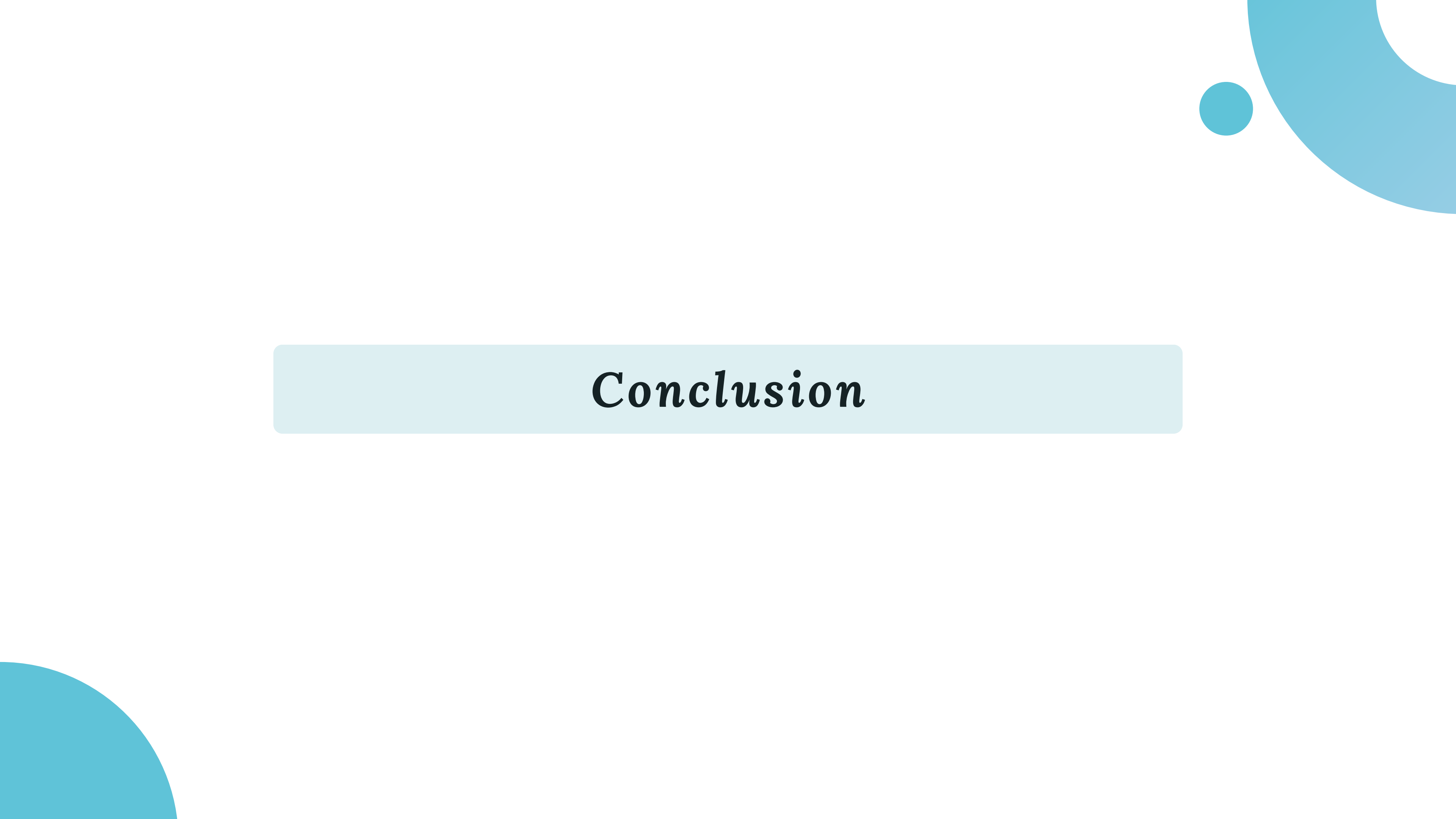
Error Analysis

- The Seq2Seq-LSTM+ and Transformer+ model outperformed the baseline Seq2Seq-LSTM and Transformer model in case of sensitive errors.
- Sensitive errors involving English words that are transliterations of Myanmar curse words and hate speech, such as “thet”, “that”, “lee”, and “darr”, can be corrected.
- The Seq2Seq-LSTM+ and Transformer+ models also decreased the overcorrection problem of Myanmar transliteration words.

- Example:

	Transformer	Transformer+
နစ် ပါ မိုင်း ရပ်စ်	နှစ် ပါ မိုင်း ရပ်စ်	နစ် ပါ မိုင်း ရပ်စ်
စင်္ကာ ပူ	စင် က ပူ	စင်္ကာ ပူ

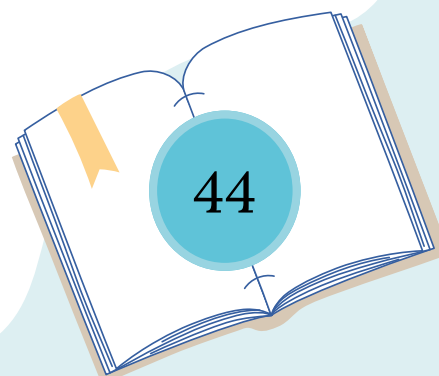




Conclusion

Conclusion

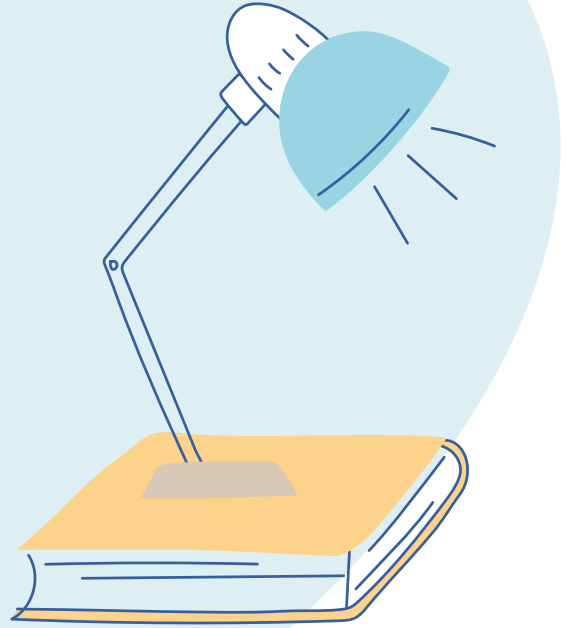
- This study contributes by:
 - Creating a parallel corpus for Myanmar spelling error correction
 - Applying Seq2Seq with LSTM and Transformer architectures
 - Exploring the impact of integrating spelling error types as linguistic features and applying data augmentation
- Our ongoing efforts include the release of the developed spelling error corpus, comprising 63,036 sentence pairs, to benefit the Myanmar NLP community.



References

1. Ei Phyu Phyu Mon et al. “SymSpell4Burmese: Symmetric Delete Spelling Correction Algorithm (SymSpell) for Burmese Spelling Checking”, 2021.
2. Ye Kyaw Thu et al. “sylbreak4all: Regular Expression based Syllable Breaking Tool for Nine Major Ethnic Languages of Myanmar”, 2021.
3. Ole-Christoffer Granmo, “The Tsetlin Machine - A Game Theoretic Bandit Driven Approach to Optimal Pattern Recognition with Propositional Logic”, 2018.
4. Armand Joulin et al. “Bag of Tricks for Efficient Text Classification”, 2016.
5. Armand Joulin et al. “Long Short-Term Memory”, 1997.
6. Ilya Sutskever et al. “Sequence to sequence learning with neural networks”, 2014.
7. Ashish Vaswani et al. “Attention Is All You Need”, 2017.
8. Guillaume Klein et al. “OpenNMT: Open-Source Toolkit for Neural Machine Translation”, 2017.
9. Rico Sennrich et al. “Linguistic Input Features Improve Neural Machine Translation”, 2016.
10. Jordi Armengol-Estapé et al. “Enriching the Transformer with Linguistic Factors for Low-Resource Machine Translation”, 2021.





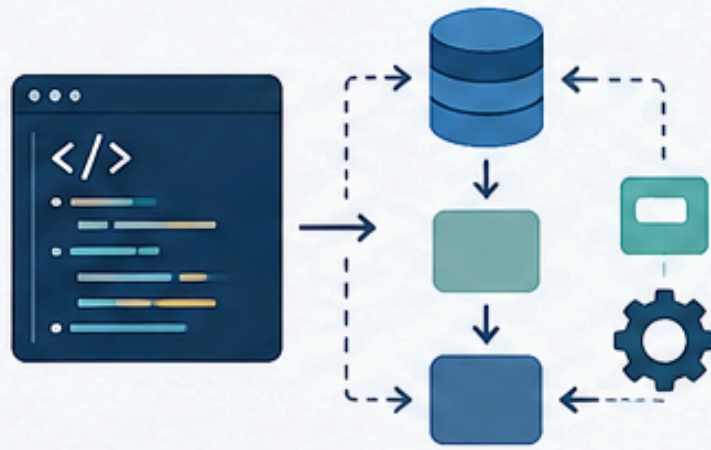
Beyond Research

Lessons from My Industry Experience



Navigating Real-World Development

1 Understand the System, Not Just the Code



- Understand the fundamentals behind the system — algorithms, databases, architecture, dependencies, data flow, and how components interact.

2 Think About Optimization & Scalability



- Don't stop at "it works."
- Consider performance, bottlenecks, resource usage, and what happens as users or data grow.

3 Think About What Can Break, Not Just What Works



- Anticipate failure scenarios and edge cases — invalid inputs, unavailable dependencies, unexpected traffic, or system failures.
- Build systems that can detect, handle, and recover from problems.

4 Communicate & Collaborate



- Clarify requirements, ask the right questions.
- Explain technical decisions and trade-offs.
- Document key decisions.
- Communicate effectively with both technical and non-technical stakeholders.

Navigating Real-World Development

1



Too Many Choices, Rapidly Changing Technology

- New models, frameworks, and tools appear constantly.
- It's challenging to decide what is worth testing and where to focus.



I learned to **focus** on what is suitable for the problem and our constraints.

2



Models Perform Differently in Real-World Environments

- Model performance can vary depending on hardware, system resources, and deployment environments.
- A high-performing model may not always be practical under real-world constraints.



I learned to **evaluate** models under realistic system conditions early, balancing performance with practical requirements.

3



Things Don't Always Work as Expected

- A promising approach can fail or perform worse than expected.
- It takes many iterations to reach a useful result.



I learned to treat failures as part of the process and **learn from them**.

4



Knowing When to Move Forward

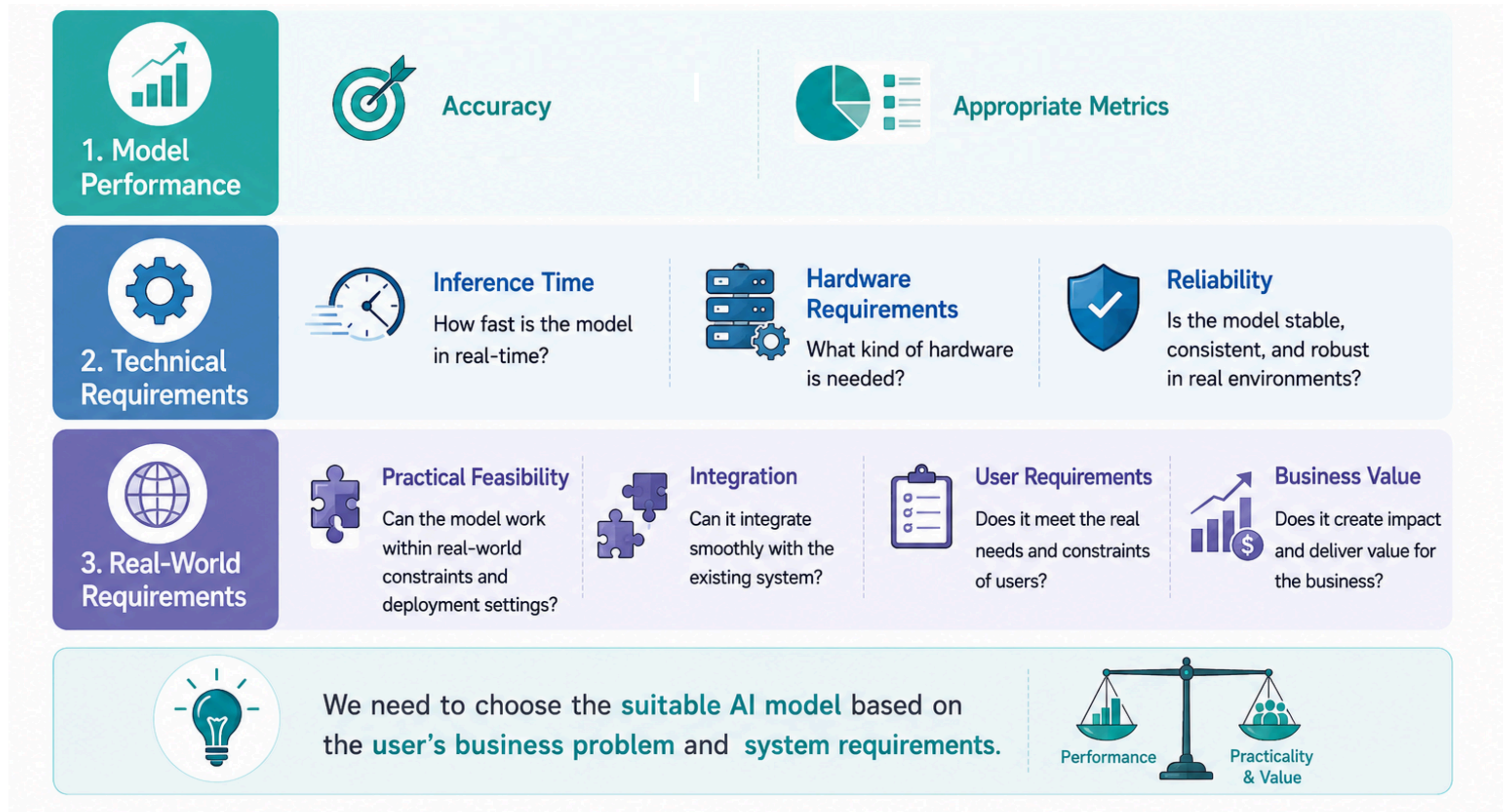
- There is always another model or approach to explore.
- In real projects, time and resources are limited, so we need to make a decision and move forward.



I learned to make decisions with enough evidence, then **monitor**, **evaluate**, and **improve** when needed.



Evaluating AI for Real-World Applications



Lessons I Carry Forward

- *Understand the problem before choosing the technology.*
- *Data matters as much as the model – understand its quality, limitations, and biases.*
- *Failure is information – learn from what doesn't work.*
- *Keep your work traceable and reproducible – document experiments, configurations, and results.*
- *Communicate AI limitations and uncertainties clearly – set realistic expectations about what a model can and cannot do.*
- *Stay updated with the latest AI trends, research, tools, and frameworks.*
- *Technologies will change – keep learning, adapting, and building on what you already know.*



Thank you!